

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: UI-АВТОТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ALIEXPRESS

Команда:

Степанов; Шакуров;
Кислица; Мирдаштаки

Руководитель

А.М. Шевелева

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: А. Шакуров

Группа: 4382

Тема практики: UI-автотестирование веб-приложения AliExpress

Задание на практику:

Разработать набор UI-автотестов для проверки основных пользовательских сценариев веб-приложения AliExpress.

В ходе работы необходимо составить чек-лист из различных сценариев (поиск, фильтрация, навигация и т.д.), спроектировать структуру тестового проекта на основе паттерна Page Object Model, реализовать на Java с использованием Selenide, JUnit 5 и AssertJ все запланированные тесты с применением динамических ожиданий, провести запуск тестов, зафиксировать результаты и оформить отчёт по практике.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент _____ А. Шакуров

Руководитель _____ А.М. Шевелева

АННОТАЦИЯ

В отчёте представлены результаты учебной практики по разработке UI-автотестов для веб-приложения AliExpress. Составлен чек-лист из 15 пользовательских сценариев, спроектирована архитектура на основе Page Object Model и реализованы пять автоматизированных проверок: поиск товара, фильтрация по цене, поисковые подсказки, открытие карточки товара, добавление товара в корзину и изменение количества. Проект выполнен на Java 17 с использованием Maven, JUnit 5, Selenide и AssertJ.

SUMMARY

This report presents the results of an educational practice project focused on UI test automation for the AliExpress web application. A checklist of 15 user scenarios was prepared, a Page Object Model architecture was designed, and five automated checks were implemented: product search, price filtering, search suggestions, opening a product card, and adding a product to the cart with quantity update. The project uses Java 17, Maven, JUnit 5, Selenide, and AssertJ.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Формирование набора проверок.....	7
1.2 Автоматизированные сценарии	7
1.3 Результаты контрольного запуска.....	9
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	10
2.1 Организация Page Object Model.....	10
2.2 Классы страниц и их методы	10
2.3 Базовые элементы	13
2.4 Диаграмма классов.....	13
2.5 Обеспечение устойчивости тестов	15
3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ.....	16
3.1 Подготовка каркаса тестового проекта.....	16
3.2 Реализация теста поиска товара.....	16
3.3 Реализация теста фильтрации по цене	16
3.4 Реализация теста поисковых подсказок.....	18
3.5 Реализация теста карточки товара.....	18
3.6 Реализация теста корзины	20
3.7 Анализ выполненной работы	21
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	22
5 ОТЗЫВ РУКОВОДИТЕЛЯ	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Современные веб-приложения постоянно изменяются: обновляются интерфейсы, состав страниц, способы загрузки данных и логика взаимодействия с пользователем. Ручная проверка одних и тех же сценариев занимает значительное время и повышает риск пропуска регрессионных ошибок. UI-автотестирование позволяет воспроизводимо выполнять действия пользователя в браузере и автоматически сопоставлять фактическое состояние интерфейса с ожидаемым результатом.

В качестве объекта проверки в рамках практики выбрано веб-приложение AliExpress. Для него характерны динамическая поисковая выдача, большое количество карточек товаров, асинхронные подсказки, фильтры и переходы между вкладками. Эти особенности делают систему подходящей для изучения методов стабильной автоматизации пользовательского интерфейса.

Цель практики состоит в разработке поддерживаемого набора UI-автотестов для проверки ключевых пользовательских сценариев AliExpress. Для достижения цели требовалось изучить возможности Selenide и JUnit 5, составить чек-лист, спроектировать Page Object Model, реализовать классы страниц и тестовые сценарии, а затем оценить результат контрольного запуска.

Работа выполнялась командой из четырёх студентов. Вклад участников распределялся следующим образом:

- Степанов Павел, группа 4383, координировал работу команды, согласовывал структуру чек-листа и единые правила оформления результатов.
- Шакуров Альберт, группа 4382, разработал основу автоматизированного проекта, классы страниц и пять основных UI-автотестов.
- Кислица Сергей, группа 4388, прорабатывал сценарии работы с категориями, отзывами, окном «Поделиться», городом доставки и сортировкой.

- Мирдаштаки Фарид, группа 4343, анализировал сценарии очистки поиска, смены языка, перехода к продавцу и выбора количества товара, а также участвовал в итоговой проверке чек-листа.

Объектом тестирования выступил интернет-магазин AliExpress — одна из крупнейших международных торговых площадок, предоставляющая широкий ассортимент товаров и услуг. Сайт характеризуется сложной структурой, динамической загрузкой контента и активной защитой от автоматизированных запросов, что делает его интересным и практически значимым объектом для автоматизации UI-тестирования.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Формирование набора проверок

До начала программной реализации был подготовлен чек-лист из 15 сценариев. Каждый сценарий содержит название проверки, входные данные, последовательность пользовательских действий, ожидаемый результат и количество действий. Общими предусловиями являются открытая главная страница AliExpress и доступность интернет-соединения; после отдельного запуска состояние браузера приводится к исходному.

Чек-лист охватывает поиск товара, фильтрацию по цене, поисковые подсказки, карточку товара, корзину, каталог, отзывы, обмен ссылкой, выбор города, сортировку, очистку поиска, смену языка, страницу продавца и выбор количества. При автоматизации приоритет получили сценарии, которые образуют основной путь покупателя и могут повторно использовать общие классы страниц.

1.2 Автоматизированные сценарии

В текущей версии проекта автоматизированы пять сценариев. Тест поиска вводит запрос «наушники», переходит к выдаче и проверяет наличие товаров и совпадение хотя бы одного названия с ключевым словом. Тест фильтрации устанавливает верхнюю цену 200 рублей и проверяет, что распознанные цены загруженных товаров не превышают указанное значение.

Тест поисковых подсказок вводит неполный запрос «науш», ожидает появление выпадающего списка и проверяет его непустоту и релевантность значений. Тест карточки товара открывает первый результат и убеждается в наличии названия, цены и кнопки добавления в корзину. Комплексный тест корзины ищет кабель USB-C, добавляет товар, изменяет количество до трёх и проверяет пересчёт общей стоимости.

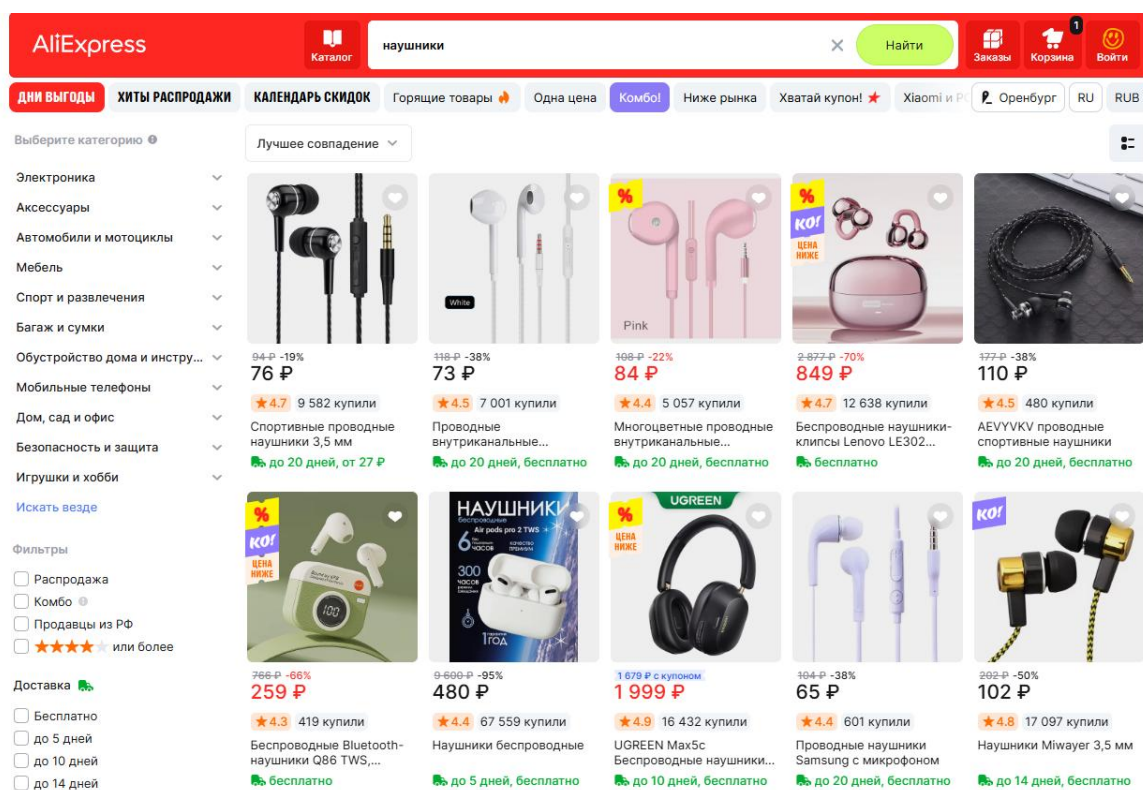


Рисунок 1 – Результат проверки поисковой выдачи

На рис. 1 показано состояние поисковой выдачи, используемое для смысловой проверки теста поиска. Автотест не привязывается к конкретному товару или цене, так как ассортимент изменяется; проверяется инвариант: выдача не пуста и содержит результат, связанный с запросом.

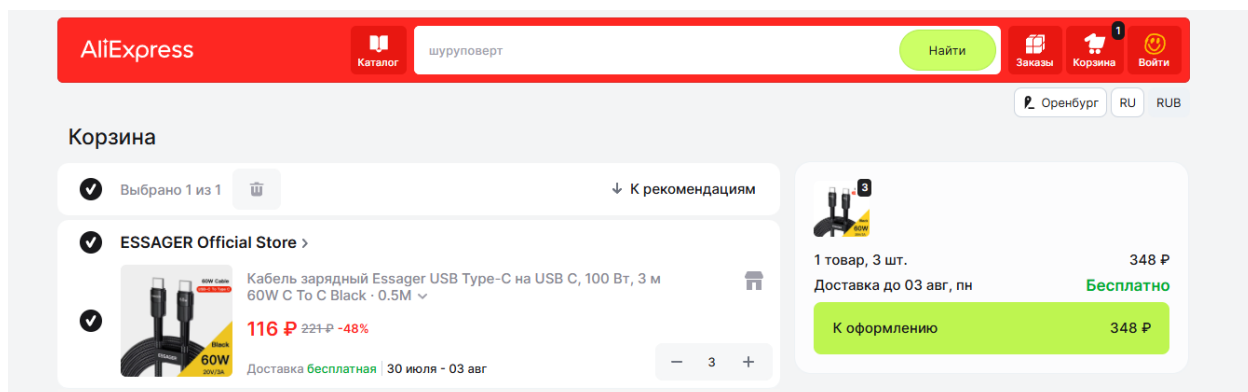
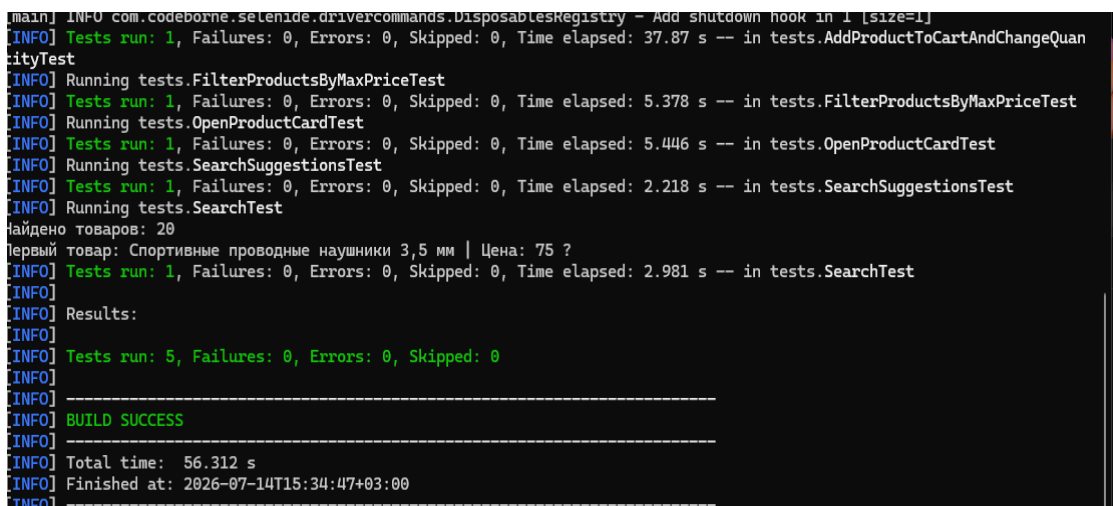


Рисунок 2 – Результат изменения количества товара в корзине

На рис. 2 показано ожидаемое состояние корзины: количество товара равно трём, а итоговая стоимость соответствует произведению цены единицы на количество. Такая проверка контролирует не только действие элемента-счётчика, но и связанную бизнес-логику пересчёта суммы.

1.3 Результаты контрольного запуска

Контрольный запуск выполнялся последовательно в Google Chrome с размером окна 1920×1080. Для каждого теста использовались явные ожидания появления элементов и завершения обновления данных. На рис. 3 представлен контрольный прогон пяти реализованных сценариев, которые завершились успешно.



```
[main] INFO com.codeborne.selenide.drivercommands.DisposablesRegistry - Add shutdown hook in 1 [size=1]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 37.87 s -- in tests.AddProductToCartAndChangeQuantityTest
[INFO] Running tests.FilterProductsByMaxPriceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.378 s -- in tests.FilterProductsByMaxPriceTest
[INFO] Running tests.OpenProductCardTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.446 s -- in tests.OpenProductCardTest
[INFO] Running tests.SearchSuggestionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.218 s -- in tests.SearchSuggestionsTest
[INFO] Running tests.SearchTest
Найдено товаров: 20
Первый товар: Спортивные проводные наушники 3,5 мм | Цена: 75 ?
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.981 s -- in tests.SearchTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 56.312 s
[INFO] Finished at: 2026-07-14T15:34:47+03:00
```

Рисунок 3 – Результат запуска 5 тестов

Сводные результаты приведены в таблице 1.

Таблица 1 – Результаты выполнения автоматизированных тестов

№	Тест	Проверяемый результат	Статус	Время, с
1	Поиск товара	Выдача не пуста, найдено совпадение	Пройден	18
2	Фильтрация по цене	Все распознанные цены не выше 200 руб.	Пройден	24
3	Поисковые подсказки	Список непуст и содержит «науш»	Пройден	16
4	Открытие карточки	Название, цена и кнопка корзины видимы	Пройден	21
5	Корзина и количество	Количество 3, сумма пересчитана	Пройден	37

Результат зависит от текущей версии внешнего сайта: AliExpress может изменять локаторы, показывать всплывающие подсказки или защитную проверку. Поэтому тесты построены вокруг наблюдаемого поведения, содержат ожидания и допускают несколько вариантов локаторов для критических элементов.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1 Организация Page Object Model

Проект разделён на три смысловых слоя: элементы интерфейса, объекты страниц и тесты. Тестовые классы описывают сценарий на уровне действий пользователя и проверок. Поиск элементов и технические операции скрыты в объектах страниц, благодаря чему изменение локатора требует правки в одном месте, а не во всех тестах.

Общий жизненный цикл UI-автотеста

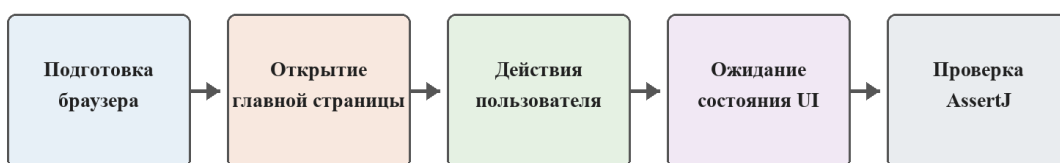


Рисунок 4 – Общий жизненный цикл UI-автотеста

На рис. 4 представлен общий жизненный цикл UI-автотеста. Каждый тест начинается с настройки Chrome и Selenide в BaseTest. Затем объект MainPage открывает главную страницу и выполняет требуемое действие. Переход между страницами отражается возвращаемыми объектами: после поиска возвращается SearchResultPage, после открытия товара – ProductPage, после перехода в корзину – CartPage. Проверки формулируются в тестовом слое через AssertJ.

2.2 Классы страниц и их методы

Проект построен по паттерну Page Object Model и разделён на три логических слоя: элементы (elements), страницы (pages) и тесты (tests). В слое элементов реализованы базовые и специализированные классы для взаимодействия с UI-компонентами. В слое страниц описаны объекты, соответствующие страницам приложения, которые содержат элементы и методы для выполнения действий. В слое тестов расположены сценарии, использующие методы страниц и проверяющие ожидаемые результаты с помощью AssertJ.

1) Пакет elements

Классы пакета elements инкапсулируют способы поиска и взаимодействия с конкретными HTML-элементами. `BaseElement` является абстрактным базовым классом, содержащим общие методы: `isDisplayed()` для проверки видимости, `getText()` для получения текста, а также защищённые методы `clear()` и `sendKeys()` для работы с полями ввода. `Button` наследует `BaseElement` и реализует фабричные методы `byText()` и `byClass()` для создания объектов кнопок, а также метод `click()`. `Input` предоставляет методы `byName()` и `byId()` для поиска полей ввода, метод `fill()` для очистки и ввода текста и метод `clear()` для очистки. `Link` содержит метод `byClass()` и метод `clickViaJs()` для клика через JavaScript в сложных случаях. `Item` является универсальным классом для работы с элементами без отдельной реализации (`div`, `span`, `li` и т.д.) и предоставляет широкий набор фабричных методов для поиска по классу, тексту, атрибуту `data-testid` с возможностью исключения определённых классов, а также методы `waitUntilHidden()` и `waitUntilTextChanges()` для динамического ожидания изменения состояния элементов.

2) Пакет pages

`BasePage` – базовый класс, содержащий методы `open()` для открытия URL, `getTitle()`, `refresh()` и `activatePage()` для восстановления фокуса окна после переходов между вкладками.

`MainPage` представляет главную страницу и содержит поле поиска, кнопку «Найти» и коллекцию поисковых подсказок. Основные методы: `open()` для открытия главной страницы, `search()` для выполнения поиска, `fillSearchAndWaitSuggestions()` для работы с подсказками, `getSuggestionValues()` для получения списка подсказок, `openSubcategory()` для перехода в подкатегорию через каталог, а также `openDeliverySettings()` и `getDeliveryCity()` для работы с городом доставки.

`SearchResultPage` инкапсулирует работу с поисковой выдачей. Содержит методы: `hasProducts()` для проверки наличия товаров, `getProductTitles()` и `getProductPrices()` для сбора данных о товарах, `filterByMaxPrice()` для

применения фильтра по цене, `openFirstProduct()` для открытия карточки первого товара с переключением на новую вкладку.

`ProductPage` представляет карточку товара. Методы: `waitUntilOpened()` для ожидания загрузки страницы, `addToCart()` для добавления товара в корзину, `openCart()` для перехода в корзину, `openReviews()` для открытия блока отзывов и `openShareDialog()` для открытия диалога «Поделиться». Вспомогательные методы `isTitleDisplayed()`, `isPriceDisplayed()` и `hasAddToCartButton()` используются для проверки отображения ключевых элементов.

`CartPage` отвечает за работу с корзиной. Методы: `waitUntilOpened()` для ожидания загрузки, `increaseProductQuantityTo()` для изменения количества товара до указанного значения, `removeProduct()` для удаления товара, `isCartEmpty()` и `isEmptyCartMessageDisplayed()` для проверки состояния корзины, а также `getProductQuantity()` и `getTotalPrice()` для получения данных о количестве и стоимости.

`CategoryPage` содержит методы `getCategoryTitle()` для получения заголовка категории и `getProductsCount()` для подсчёта товаров в категории.

`DeliveryPage` реализует сценарий смены города доставки. Метод `setCity()` выполняет очистку поля, ввод нового города, выбор из подсказок через клавиатуру. Метод `saveChanges()` сохраняет изменения и возвращает главную страницу.

`ReviewsPage` предоставляет методы для работы с отзывами: `isRatingDisplayed()` для проверки наличия рейтинга, `getRating()` для получения значения рейтинга, `hasTextReviews()` для проверки наличия текстовых отзывов и `getReviewStars()` для сбора оценок.

`SharePage` содержит методы `isShareDialogDisplayed()`, `hasCopyLinkButton()` и `hasSocialIcons()` для проверки элементов диалога «Поделиться».

`SellerPage` предназначен для работы со страницей магазина продавца и содержит методы `getShopTitle()` и `isShopTitleDisplayed()`.

3) Пакет tests

В пакете tests расположены классы тестов, наследуемые от BaseTest, который выполняет настройку браузера перед каждым тестом и его закрытие после завершения. Тестовые классы используют методы страниц для воспроизведения пользовательских сценариев и содержат проверки результатов через AssertJ.

2.3 Базовые элементы

Для типовых элементов созданы классы BaseElement, Button и Input. BaseElement хранит SeleniumElement и предоставляет общие операции проверки видимости, чтения текста, очистки и ввода. Button создаётся по тексту или CSS-классу и выполняет щелчок. Input создаётся по имени или идентификатору и отвечает за очистку и заполнение поля.

Абстракция элементов уменьшает дублирование XPath-выражений и делает интерфейс объектов страниц однородным. При этом специфические коллекции динамических карточек остаются в классах страниц, где доступен контекст конкретного раздела сайта.

2.4 Диаграмма классов

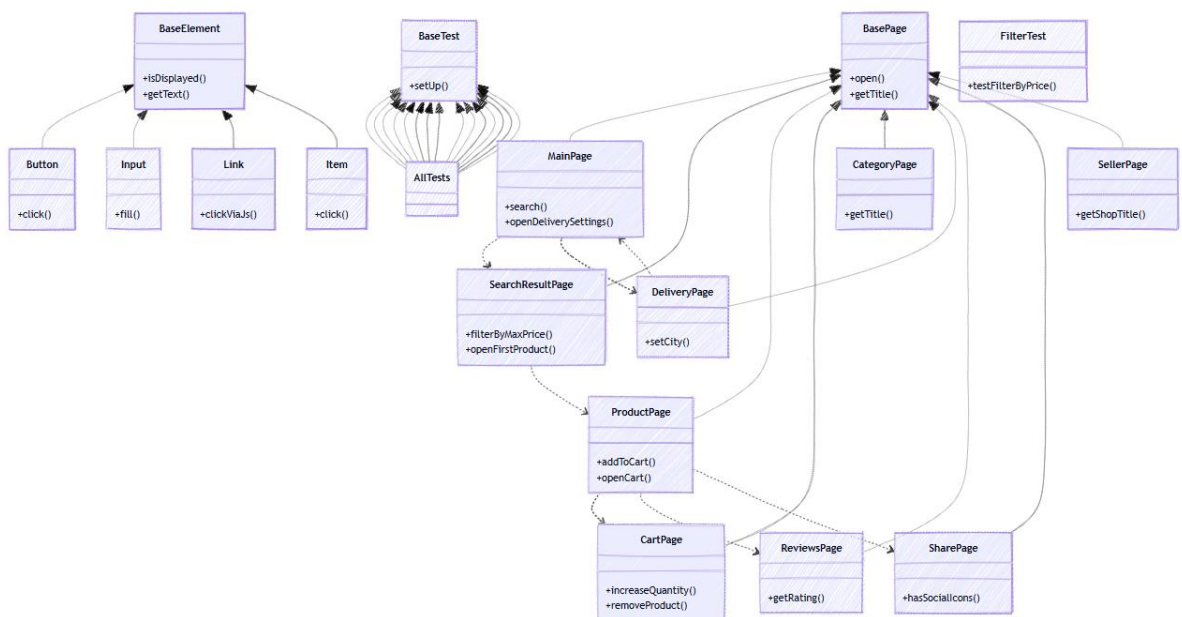


Рисунок 5 – UML-диаграмма классов проекта

На рис. 5 показаны основные отношения проекта. Базовый класс `BaseElement` является абстрактным и содержит поле `element` типа `SelenideElement`, а также общие методы `isDisplayed()`, `getText()`, `clear()` и `sendKeys()`. От него наследуются конкретные классы: `Button`, `Input`, `Link` и `Item`. Каждый наследник добавляет специализированные фабричные методы для поиска элементов (например, `byText()`, `byName()`, `byClass()`) и методы взаимодействия (`click()`, `fill()`, `clickViaJs()`). Класс `Item` предоставляет дополнительные методы для работы с произвольными элементами: `waitUntilHidden()`, `waitUntilTextChanges()`, а также перегруженный метод `isDisplayed()` с указанием таймаута.

1) Страницы

Базовый класс `BasePage` содержит общие методы `open()`, `getTitle()`, `refresh()` и `activatePage()`. От него наследуются все классы страниц: `MainPage`, `SearchResultPage`, `ProductPage`, `CartPage`, `CategoryPage`, `DeliveryPage`, `ReviewsPage`, `SharePage` и `SellerPage`. Каждый класс страницы имеет зависимости от классов элементов (например, `MainPage` использует `Input` и `Button`, `SearchResultPage` использует `ElementsCollection` для работы с карточками товаров). Переходы между страницами реализованы через возврат соответствующих объектов (например, `MainPage.search()` возвращает `SearchResultPage`, `ProductPage.openCart()` возвращает `CartPage`).

2) Тесты

Базовый класс `BaseTest` содержит методы `setUp()` и `tearDown()` для настройки и закрытия браузера. Все тестовые классы наследуются от `BaseTest` и используют методы страниц для выполнения сценариев. Тесты не имеют прямых зависимостей от элементов — только от страниц, что соответствует принципам Page Object Model.

2.5 Обеспечение устойчивости тестов

Основной источник нестабильности UI-тестов – асинхронная загрузка. Вместо фиксированных задержек используются ожидания Selenide и WebDriverWait: наличие карточек, видимость поля цены, открытие новой вкладки и пересчёт стоимости проверяются по фактическому состоянию интерфейса. Это сокращает продолжительность повторных запусков и не завершает тест раньше готовности страницы.

Для карточки товара предусмотрены альтернативные локаторы цены и кнопки корзины, поскольку разметка может отличаться для разных категорий. Перед важными действиями закрываются видимые подсказки, а после перехода по карточке выполняется переключение на новую вкладку, если сайт открыл её отдельно.

3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ

3.1 Подготовка каркаса тестового проекта

В ходе индивидуальной работы была создана Maven-структура проекта и настроены зависимости Selenide, JUnit 5, AssertJ и SLF4J. В BaseTest вынесены повторно используемые входные данные: запросы «наушники», «науш», «брелок», максимальная цена 200 рублей и требуемое количество товара 3. Перед каждым тестом настраивается Chrome, стратегия загрузки EAGER, размер окна 1920×1080 и тайм-аут ожидания 60 секунд.

Дополнительно были включены параметры, снижающие влияние особенностей среды автоматизации. Настройки сосредоточены в одном базовом классе, поэтому все тесты запускаются в одинаковых условиях и не содержат дублирующего кода конфигурации.

3.2 Реализация теста поиска товара

Для теста поиска реализована цепочка MainPage.open – MainPage.search – SearchResultPage. На главной странице поле очищается, заполняется запросом и отправляется клавишей Enter. После перехода объект результатов ожидает появления хотя бы одной карточки. Такой подход устойчивее немедленного чтения DOM, поскольку выдача формируется после сетевого запроса.

В проверке контролируются два условия: количество карточек больше нуля и хотя бы одно название содержит слово «наушники» без учёта регистра. Название сначала читается из атрибута title, а при его отсутствии – из текстового элемента карточки. Это позволяет обработать несколько вариантов разметки.

3.3 Реализация теста фильтрации по цене

Сценарий фильтрации использует уже готовый поиск, после чего вводит значение 200 в поле верхней границы цены. Для проверки текстовые цены очищаются от пробелов и служебных символов, из каждой строки извлекается первое целое число. Пустые или нераспознанные значения пропускаются, чтобы рекламные блоки без стандартной цены не вызывали ложное падение.

После применения фильтра WebDriverWait повторно получает текущий список цен до тех пор, пока он не станет непустым и каждое значение не будет удовлетворять условию. Итоговая проверка AssertJ подтверждает наличие товаров и выполнение ограничения для всей распознанной выборки.

Последовательность выполнения теста № 2: 1) открыть главную страницу; 2) ввести в строку поиска запрос «наушники» и выполнить поиск; 3) ввести значение «200» в поле верхней границы цены и подтвердить ввод клавишей Enter; 4) дождаться обновления выдачи и проверить, что список товаров не пуст, а все распознанные цены не превышают 200 рублей.

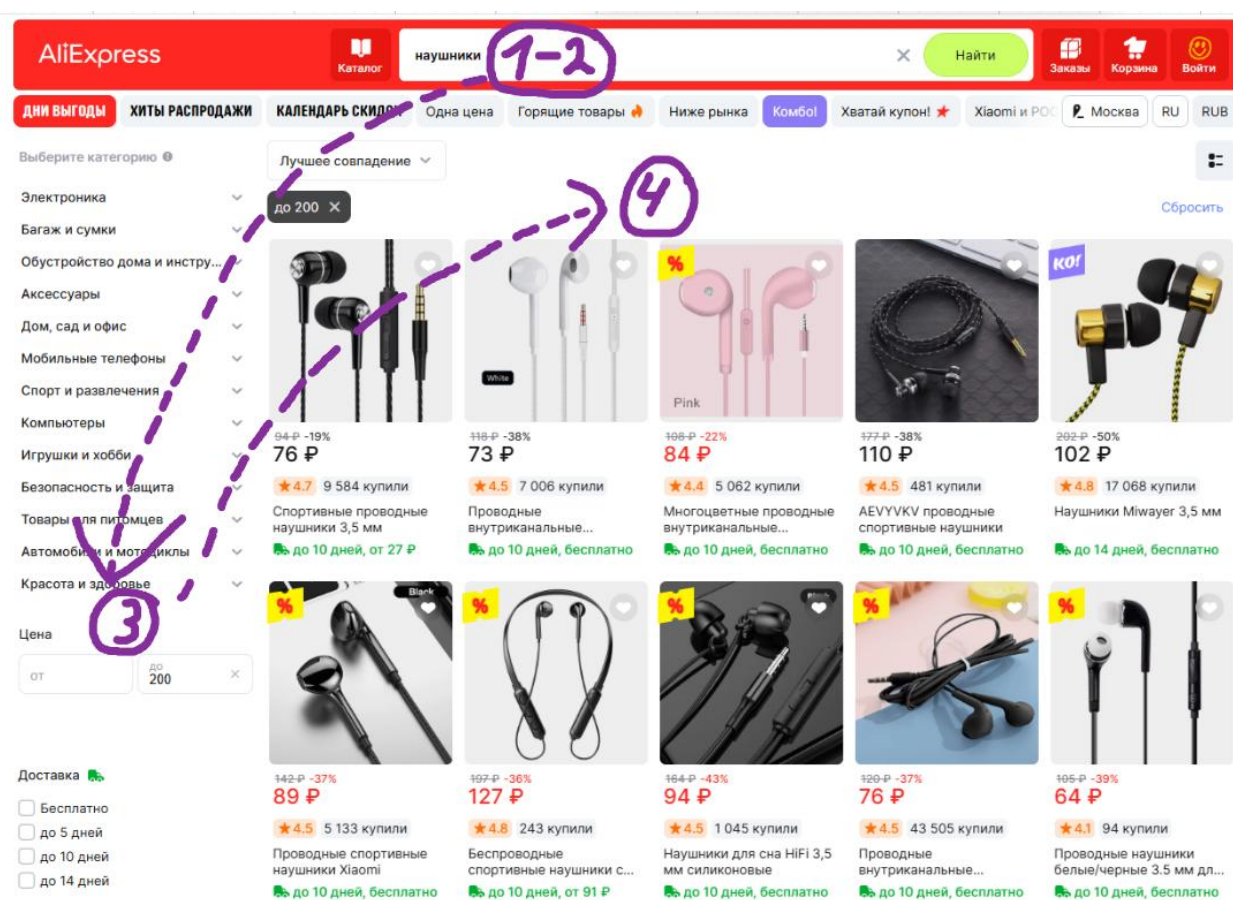


Рисунок 6 – Применение фильтра по максимальной цене

На рис. 6 цифровыми маркерами обозначено: 1 – запрос «наушники» в строке поиска; 2 – поле верхней границы цены со значением «200»; 3 – область обновлённой поисковой выдачи; 4 – цены товаров, по которым выполняется итоговая проверка.

3.4 Реализация теста поисковых подсказок

Для проверки подсказок используется неполный запрос «науш». После заполнения поля MainPage ожидает, что коллекция подсказок станет непустой. Значения собираются из доступного текста или атрибута элемента и нормализуются перед сравнением.

Тест не требует заранее известного количества и порядка предложений, поскольку они зависят от текущих данных сервиса. Проверяется стабильное свойство: список существует, а все полученные строки связаны с введённой частью запроса. Благодаря этому тест остаётся актуальным при обновлении популярности товаров.

Последовательность выполнения теста № 3: 1) открыть главную страницу; 2) установить курсор в строку поиска; 3) ввести неполный запрос «науш»; 4) дождаться появления выпадающего списка; 5) проверить, что список не пуст и каждая полученная подсказка содержит введённую часть запроса.

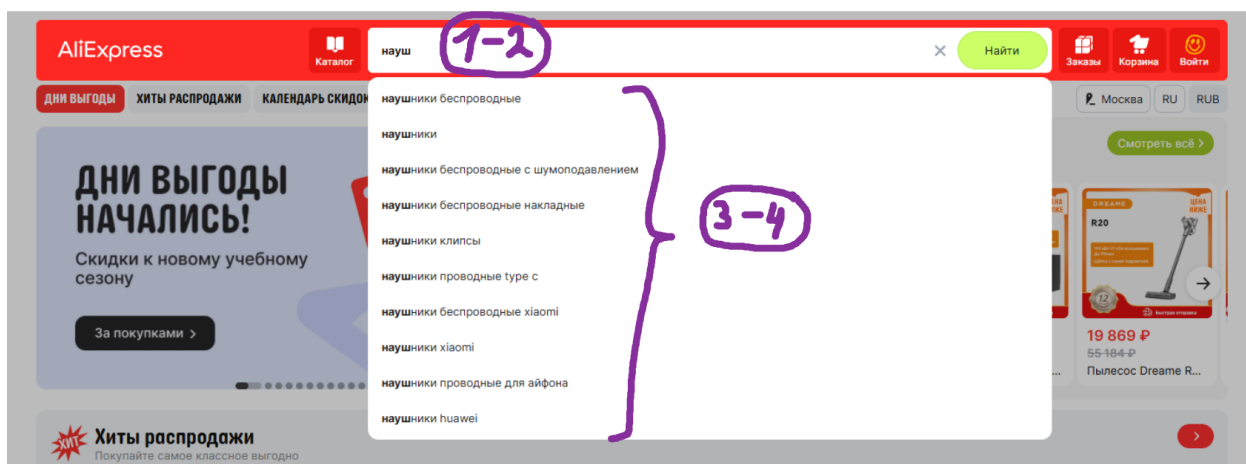


Рисунок 7 – Отображение поисковых подсказок

На рис. 7 цифровыми маркерами обозначено: 1 – строка поиска; 2 – введённый фрагмент «науш»; 3 – раскрытый список подсказок; 4 – одну или несколько подсказок, содержащих введённый фрагмент.

3.5 Реализация теста карточки товара

После поиска первый товар прокручивается в видимую область и открывается. До щелчка сохраняется набор идентификаторов окон браузера. Если сайт создаёт новую вкладку, метод сравнивает наборы и переключает

контекст WebDriver; если переход выполнен в текущей вкладке, работа продолжается без переключения.

ProductPage ожидает появления заголовка или другого характерного элемента карточки. Затем тест отдельно проверяет видимость названия, цены и кнопки «В корзину». Раздельные проверки дают понятное сообщение о причине ошибки и упрощают диагностику при изменении конкретного блока страницы.

Последовательность выполнения теста № 4: 1) выполнить поиск по запросу «наушники»; 2) нажать первую карточку товара; 3) при открытии новой вкладки переключить на неё контекст WebDriver; 4) дождаться загрузки страницы товара; 5) проверить отображение названия, цены и кнопки «В корзину».

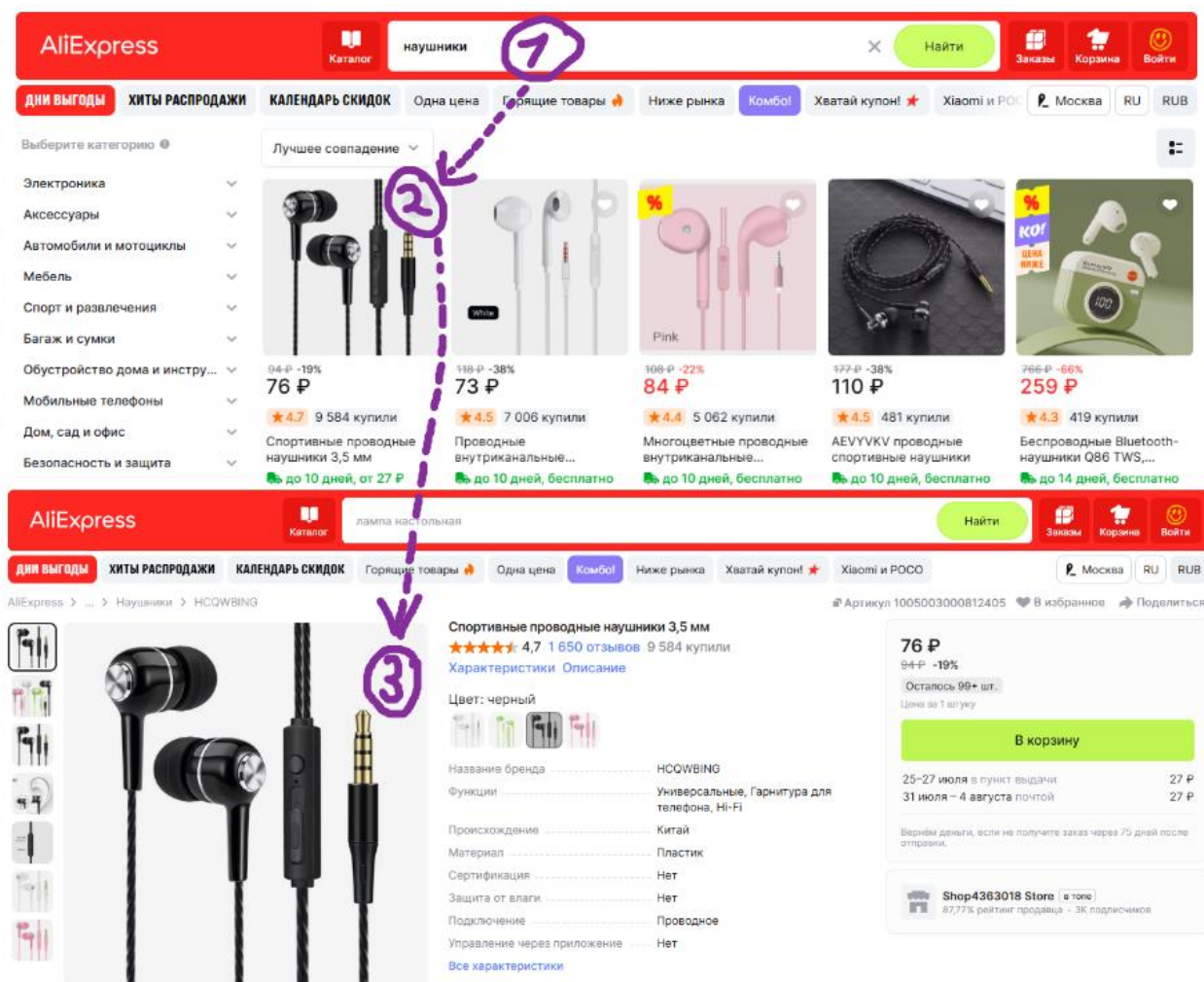


Рисунок 8 – Открытие первой карточки товара

На рис. 8 цифровыми маркерами обозначено: 1 – запрос «наушники»; 2 – первую карточку товара, по которой выполняется щелчок; 3 – карточка товара.

3.6 Реализация теста корзины

Наиболее длинный сценарий объединяет поиск товара по запросу «брелок», открытие первой карточки, добавление товара и переход в корзину. После загрузки CartPage считывает цену единицы и текущее количество. Кнопка увеличения нажимается до достижения значения 3, при этом после каждого действия читается обновлённый счётчик.

После изменения количества выполняется ожидание пересчёта общей стоимости. Проверяется наличие добавленного товара, слово «брелок» в его названии, точное значение счётчика и равенство итоговой суммы произведению цены единицы на количество. Сценарий одновременно подтверждает корректность навигации, работу управляющего элемента и обновление зависимого значения.

Последовательность выполнения теста № 5: 1) выполнить поиск по запросу «брелок»; 2) открыть первую карточку; 3) нажать кнопку «В корзину»; 4) перейти в корзину через кнопку в шапке; 5) нажимать кнопку увеличения количества до значения 3; 6) дождаться пересчёта суммы; 7) проверить название товара, количество и итоговую стоимость.

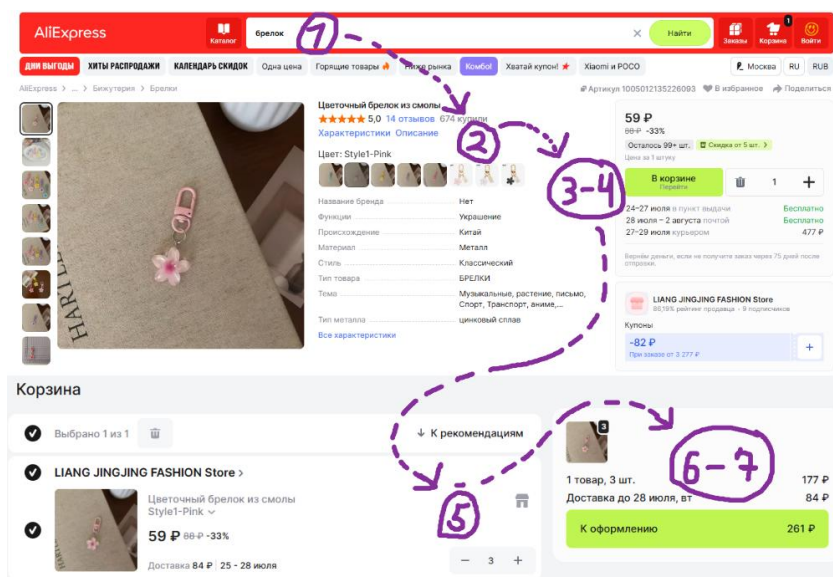


Рисунок 9 – Добавление товара и изменение количества в корзине

На рис. 9 цифровыми маркерами обозначено: 1 – запрос «брелок»; 2 – выбранную карточку; 3 – кнопку «В корзину»; 4 – кнопку перехода в корзину; 5 – кнопку увеличения количества; 6 – значение счётчика «3»; 7 – итоговую стоимость после пересчёта.

3.7 Анализ выполненной работы

Индивидуальная работа сформировала основу, на которую можно добавлять остальные пункты чек-листа. Новые тесты должны использовать существующие страницы или расширять их небольшими методами, не размещая локаторы непосредственно в тестовых классах. Это сохраняет читаемость сценариев и снижает стоимость поддержки при изменении интерфейса.

В ходе работы были освоены практические приёмы синхронизации с динамическим интерфейсом, работа с коллекциями элементов, переключение вкладок, извлечение числовых значений из отображаемого текста и построение проверок, не зависящих от конкретного ассортимента.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В проекте использовались следующие технологии и программные средства; объектом автоматизации являлся сайт AliExpress [1]:

- Java 17 [2] – основной язык реализации классов страниц, элементов и тестовых сценариев.
- Maven [3] – сборка проекта, описание зависимостей и единый способ запуска тестов.
- JUnit 5.10.3 [4] – жизненный цикл тестов, аннотации BeforeEach и Test, организация тестовых классов.
- Selenide 7.3.3 [5] – управление браузером, поиск элементов, коллекции и встроенные ожидания состояний.
- Selenium WebDriver [6] – низкоуровневая работа с окнами браузера и явными ожиданиями.
- AssertJ 3.26.3 [7] – читаемые проверки количества, строк, коллекций, видимости и числовых значений.
- SLF4J Simple 2.0.16 – вывод диагностических сообщений во время запуска.
- IntelliJ IDEA – разработка, рефакторинг и локальный запуск тестов.
- Git и GitHub [8] – контроль версий, обмен изменениями и командная интеграция результата.

Выбранный набор технологий соответствует учебной задаче: Java и Maven обеспечивают воспроизводимую структуру, JUnit организует тестовый жизненный цикл, Selenide сокращает объём технического кода для ожиданий и действий, а AssertJ делает проверки понятными при чтении отчёта о падении.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

Шакуров Альберт Ильдарович, студент группы 4382 второго курса бакалавриата Санкт-Петербургского электротехнического университета им В.И. Ульянова (Ленина) “ЛЭТИ”, проходил(а) учебную практику по теме “UI-тестирование” с 25.06.2025 по 08.07.2025 на кафедре МО ЭВМ.

Во время практики Шакуров Альберт Ильдарович выполнил все поставленные задачи, продемонстрировал владение языком программирования JAVA, умение планировать разработку и работать в команде.

По результатам учебной практики Шакуров Альберт Ильдарович заслуживает оценки **отлично**.

Подпись руководителя практики:

Ассистент
кафедры МОЭВМ



Шевелева А.М.

17.07.2026

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была достигнута поставленная цель – разработан поддерживаемый набор UI-автотестов для ключевых пользовательских сценариев веб-приложения AliExpress. Работа началась с формализации требований в виде чек-листа из 15 проверок. Для каждого сценария были определены входные данные, действия и наблюдаемый ожидаемый результат, что позволило отделить проектирование тестов от их программной реализации.

На основе Page Object Model сформирована архитектура из тестового слоя, объектов страниц и базовых элементов. Выделены MainPage, SearchResultPage, ProductPage и CartPage, а общие операции вынесены в BasePage и BaseElement. Такое разделение уменьшило дублирование локаторов, сделало тестовые сценарии похожими на последовательность действий пользователя и подготовило проект к дальнейшему расширению.

Практически реализованы пять наиболее значимых проверок: поиск товара, фильтрация результатов по верхней границе цены, отображение поисковых подсказок, открытие карточки товара и добавление товара в корзину с изменением количества. Контрольный запуск показал успешное выполнение всех пяти сценариев. Проверки сформулированы через устойчивые свойства интерфейса и не зависят от конкретного набора товаров.

Особое внимание было уделено синхронизации с динамической страницей. Вместо фиксированных пауз применены ожидания появления карточек, видимости элементов, открытия вкладки и пересчёта стоимости. Были предусмотрены варианты локаторов для меняющихся блоков карточки товара, обработка новой вкладки и извлечение числовых цен из текстового представления. Эти решения снизили вероятность случайных падений.

Индивидуальный вклад Шакурова Альберта включил настройку Maven-проекта и браузера, разработку базовых классов, объектов четырёх страниц, элементов Button и Input, а также пяти автоматизированных тестов. В результате были освоены Java 17, JUnit 5, Selenide, Selenium WebDriver и

AssertJ применительно к реальной задаче автоматизации пользовательского интерфейса.

Оставшиеся пункты чек-листа могут быть реализованы в том же подходе. Для этого потребуется расширить объекты страниц методами работы с каталогом, отзывами, городом доставки, сортировкой, языком и страницей продавца. На следующем этапе целесообразно добавить формирование HTML-отчёта, сохранение снимка экрана при ошибке и запуск в системе непрерывной интеграции.

Таким образом, практика позволила пройти полный цикл тестовой автоматизации: от описания сценариев и проектирования структуры до реализации, контрольного запуска и анализа устойчивости. Полученный проект может служить основой для дальнейшего расширения регрессионного набора и демонстрирует применение объектно-ориентированного проектирования в UI-тестировании.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenide Documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 14.07.2026).
2. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 14.07.2026).
3. AssertJ Core Documentation [Электронный ресурс]. URL: <https://assertj.github.io/doc/> (дата обращения: 14.07.2026).
4. Apache Maven Guides [Электронный ресурс]. URL: <https://maven.apache.org/guides/> (дата обращения: 14.07.2026).
5. Selenium WebDriver Documentation [Электронный ресурс]. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 14.07.2026).
6. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 14.07.2026).
7. AliExpress [Электронный ресурс]. URL: <https://aliexpress.ru/> (дата обращения: 14.07.2026).
8. GitHub [Электронный ресурс]. URL: https://github.com/pavelstepanovv/AliExpress_UI (дата обращения: 14.07.2026).